Documentation Projet CyberScale v1.0.1

Table of Contents

1. Présentation . . . 1
   1.1. L'Équipe . . . 2
   1.2. Vision et Objectifs (v1.0.1) . . . 2
2. Architecture et Technique . . . 2
   2.1. Architecture Détaillée (v1.0.1) . . . 2
      2.1.1. Vue Infrastructure : Orchestration & Messages (Nouveau) . . . 2
      2.1.2. Vue Logique : Traitement & Services . . . 3
      2.1.3. Vue de Données : Persistance . . . 3
   2.2. Stack Technique . . . 3
   2.3. Focus : La Dockerisation & Flags Dynamiques . . . 4
3. Utilisation de l'IA Générative . . . 4
4. Fonctionnalités (Features v1.0.1) . . . 5
   4.1. F1: Onboarding & Quiz Adaptatif . . . 5
   4.2. F2: Cyber Arena (Red Team) - **Amélioré v1.0.1** . . . 6
   4.3. F3: Module Phishing (Facteur Humain) . . . 7
   4.4. F4: Module Investigation (Blue Team) . . . 7
   4.5. F5: Dashboard Unifié & Certification . . . 7
   4.6. F6: Asynchronisme & Robustesse (RabbitMQ) - **Nouveau v1.0.1** . . . 8
   4.7. F7: Défense Active & Leurres (Kubernetes) - **Nouveau v1.0.1** . . . 8
5. Gestion de Projet & Qualité . . . 8
   5.1. Sécurité & Authentification . . . 8
   5.2. Qualité du Code (CI/CD) . . . 8
6. Conclusion & Perspectives . . . 9

Documentation technique et fonctionnelle de la plateforme CyberScale. *Cette version v1.0.1 apporte une stabilisation de l'infrastructure avec l'intégration de RabbitMQ pour l'asynchronisme et une meilleure orchestration Docker/Kubernetes.*

1. Présentation

1.1. L’Équipe

Projet réalisé dans le cadre du cursus DevOps par :

- **Lyes SEHILA** - Lead DevOps & Architecte
- **Hassan Jatta** - Lead Backend
- **Abdoulaye** - Lead Frontend

1.2. Vision et Objectifs (v1.0.1)

CyberScale a pour mission de fournir une évaluation "360 degrés" des compétences en cybersécurité. La version v1.0.1 met l’accent sur la résilience et la scalabilité de l’infrastructure.

2. Architecture et Technique

2.1. Architecture Détaillée (v1.0.1)

Pour garantir une meilleure lisibilité, l’architecture est présentée en trois vues distinctes.

2.1.1. Vue Infrastructure : Orchestration & Messages (Nouveau)

Ce diagramme illustre le flux de données entre le frontend, le backend et les services d’infrastructure (RabbitMQ, Docker, Kubernetes).

- **RabbitMQ** : Gère les files d’attente pour l’exécution asynchrone des commandes.
- **Kubernetes** : Utilisé pour le déploiement dynamique de leurres (honeypots).

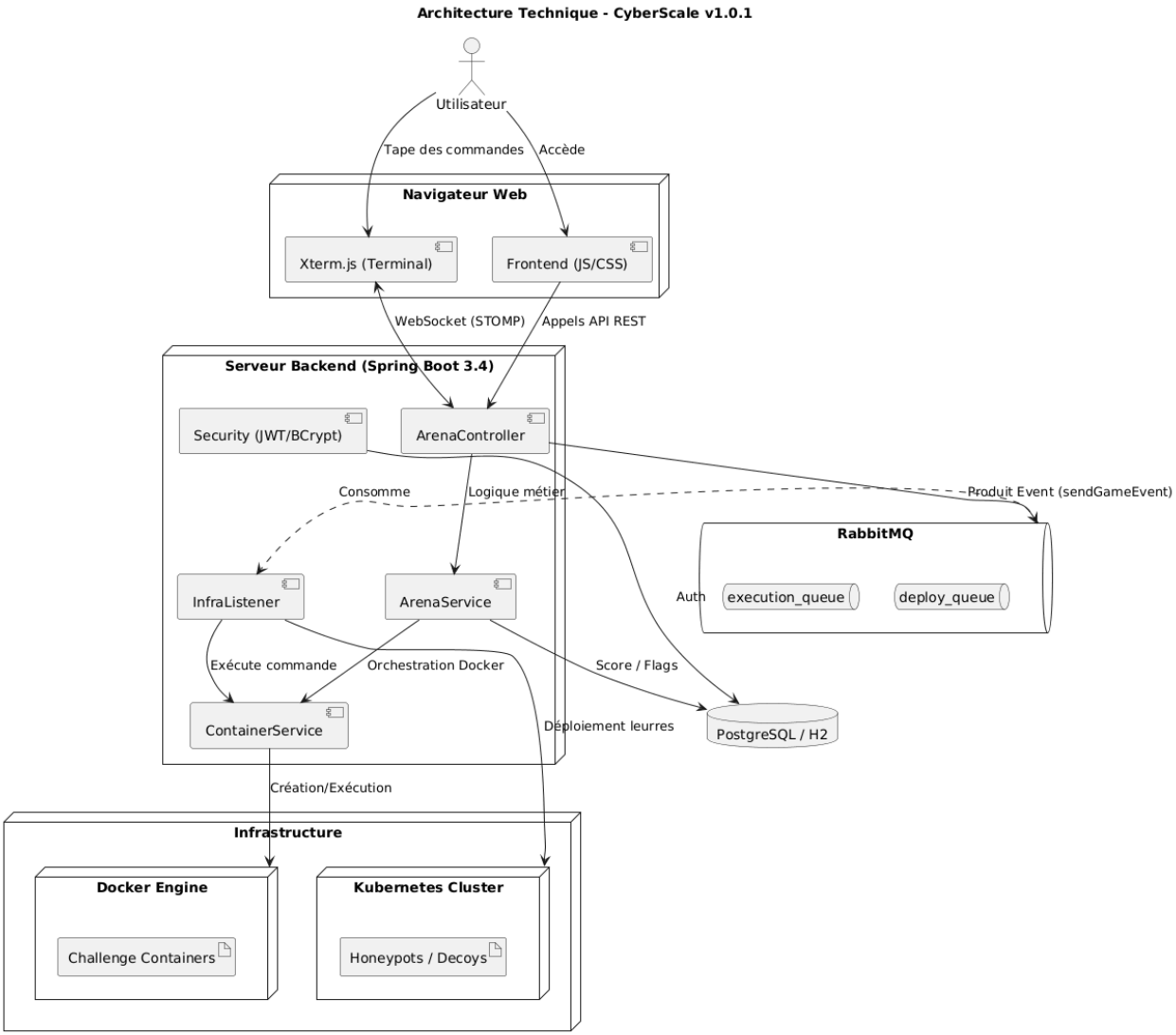

Figure 1. Architecture Technique Globale (v1.0.1)

2.1.2. Vue Logique : Traitement & Services

Ce diagramme illustre la couche "Intelligente" de CyberScale. On y voit comment les Contrôleurs (API) orchestrent les Services métiers. Notez particulièrement l'interaction entre `ArenaController` et le `ContainerService` (gestion Docker), ainsi que le générateur de logs pour la Blue Team.

2.1.3. Vue de Données : Persistance

Ce diagramme présente la structure de la base de données PostgreSQL (Entités JPA). Il met en évidence la centralisation des données autour de l'entité `User`, qui est liée à ses résultats de quiz, ses challenges réussis et ses examens.

2.2. Stack Technique

Domaine	Technologies
Backend	Java 21, Spring Boot 3.4.11 (Security, JPA, WebSocket)
Données	PostgreSQL 16 (Production), H2 (Développement)

Domaine	Technologies
Messaging	**RabbitMQ 3** (Management)
Orchestration	**Docker Engine**, **Kubernetes** (Honeypots)
Frontend	HTML5, CSS3, JavaScript (Vanilla), Tailwind CSS, Xterm.js
DevOps	GitHub Actions (CI/CD), SonarCloud, JaCoCo
Tests	JUnit 5, Selenium (E2E), Cucumber (BDD), Mockito

2.3. Focus : La Dockerisation & Flags Dynamiques

C'est l'innovation technique majeure de cette release. Pour permettre aux utilisateurs de s'entraîner sans risque, nous avons développé un orchestrateur de conteneurs interne.

- **Flags Dynamiques :** Contrairement à la v1.0.0, chaque conteneur possède désormais un flag unique généré à la volée (`CHALLENGE_FLAG`), empêchant le partage de solutions.
- **Isolation Totale :** Chaque challenge technique lance un conteneur Docker dédié à l'utilisateur.
- **Communication :** Un tunnel **WebSocket** relie le terminal Web (`Xterm.js`) à l'entrée/sortie standard du conteneur via RabbitMQ pour une exécution robuste.

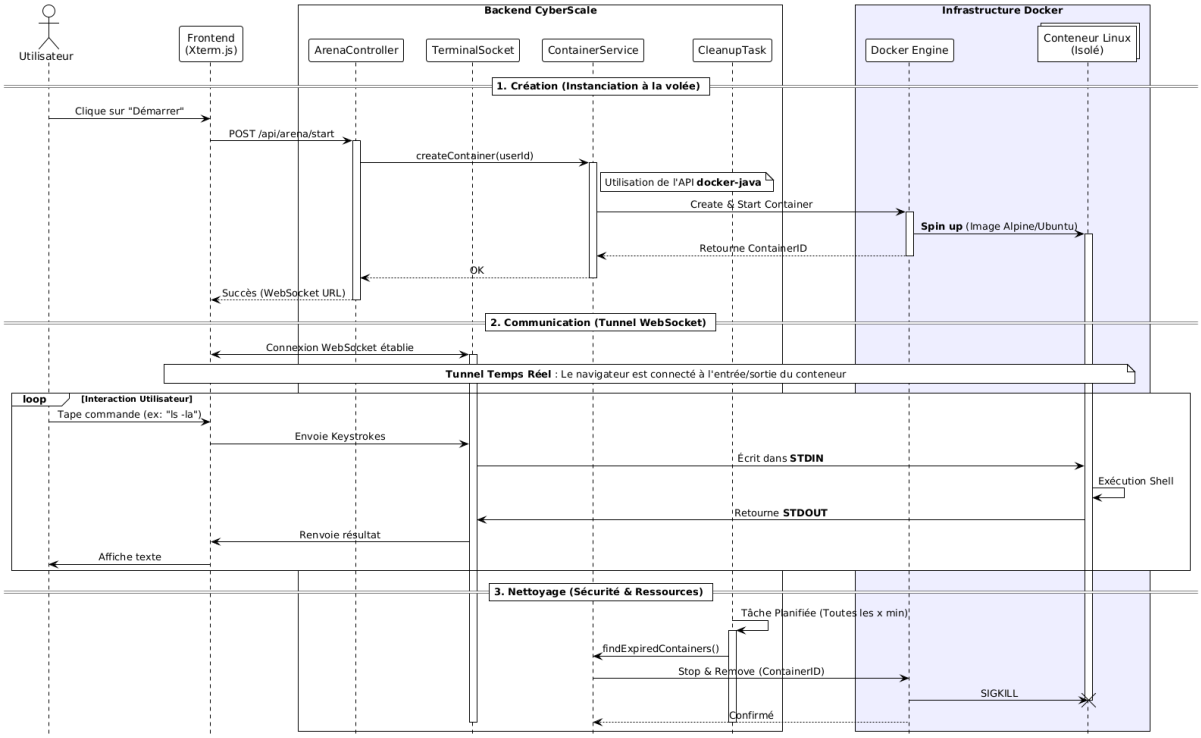

Figure 2. Fonctionnement de l'architecture dockerisé

3. Utilisation de l'IA Générative

Dans une démarche de transparence et d'optimisation du temps de développement, nous avons utilisé des outils d'Intelligence Artificielle (type LLM) pour des tâches d'assistance spécifiques, sans leur déléguer l'architecture cœur du projet.

- **Génération de Jeux de Données (Data Seeding)** : Pour le module d'Investigation (Blue Team),

il était nécessaire de créer des milliers de lignes de logs réalistes (Apache/Linux). L'IA a été utilisée pour générer ces motifs de logs et y insérer des scénarios d'attaques variés, une tâche trop répétitive pour être faite manuellement.

- **Création de Contenu Pédagogique** : Afin de peupler la base de données du Quiz, l'IA nous a assistés dans la rédaction des questions et des explications associées, nous permettant de nous concentrer sur l'algorithme de pondération plutôt que sur la rédaction.
- **Aide au Débogage Docker** : Lors de la configuration complexe des conteneurs isolés pour l'Arena, l'IA a servi d'assistant pour analyser certaines erreurs obscures liées aux permissions Linux et aux WebSockets.

4. Fonctionnalités (Features v1.0.1)

Cette version apporte des améliorations majeures sur l'infrastructure et la sécurité des épreuves.

4.1. F1: Onboarding & Quiz Adaptatif

Objectif

Évaluer le bagage théorique initial.

Innovation

L'algorithme adapte la difficulté des questions (Facile → Moyen → Difficile) en fonction des réponses précédentes pour affiner le score.

Figure 3. Wireframe Quiz

4.2. F2: Cyber Arena (Red Team) - Amélioré v1.0.1

Objectif

Tester les compétences offensives (Linux, Pentest).

Scénario

L'utilisateur accède à un vrai terminal Linux isolé par Docker.

Nouveauté v1.0.1

Flags Dynamiques. Le flag est désormais généré à la volée (`CHALLENGE_FLAG`) et stocké temporairement en mémoire pour l'utilisateur, empêchant toute triche par partage de solutions.

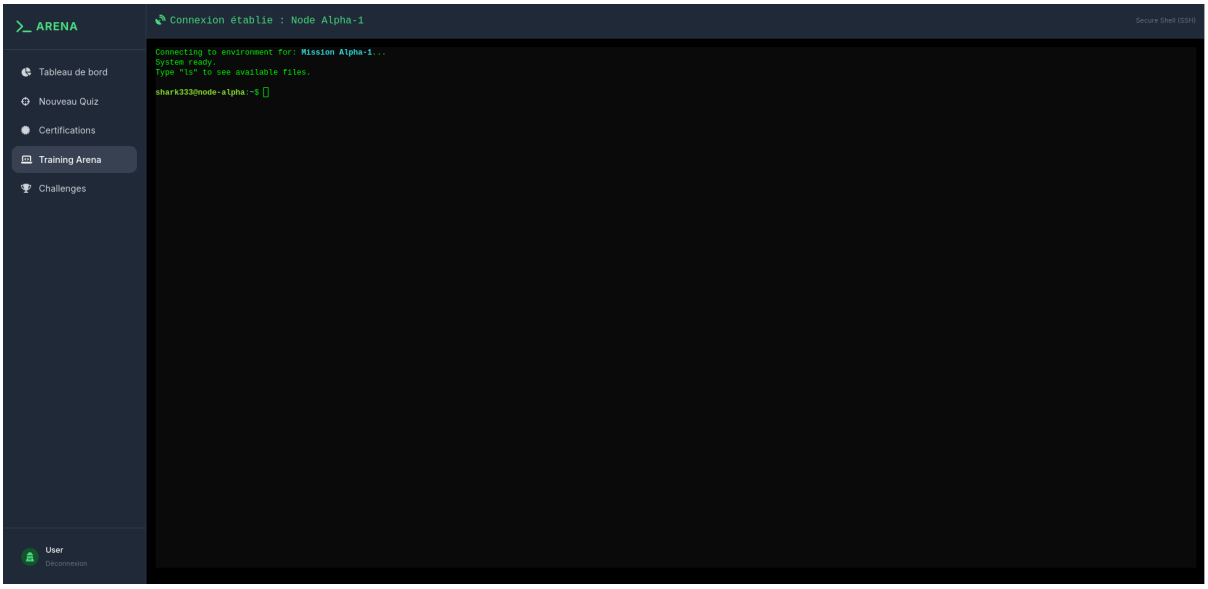

Figure 4. Interface du Terminal Web

4.3. F3: Module Phishing (Facteur Humain)

Objectif

Tester la vigilance face à l'ingénierie sociale.

Fonctionnement

Simulation d'une boîte mail avec des courriels piégés (Urgence RH, Faux support). L'utilisateur doit analyser les en-têtes et décider de signaler ou cliquer.

4.4. F4: Module Investigation (Blue Team)

Objectif

Former à la défense et à l'analyse de logs (Forensics).

Technique

Un générateur de logs (`LogGenerator`) crée dynamiquement des milliers de lignes de trafic web réaliste.

4.5. F5: Dashboard Unifié & Certification

Objectif

Centraliser la progression.

Évolution

Agrégation des scores de tous les modules pour calculer une moyenne globale et débloquer des certifications.

4.6. F6: Asynchronisme & Robustesse (RabbitMQ) - Nouveau v1.0.1

Objectif

Assurer une exécution fluide et scalable des commandes.

Technique

Utilisation de **RabbitMQ** pour traiter les événements de jeu et les commandes de l'Arena de manière asynchrone, évitant les blocages du thread principal.

4.7. F7: Défense Active & Leurres (Kubernetes) - Nouveau v1.0.1

Objectif

Simuler des environnements de défense réseau complexes.

Innovation

Intégration de **KubernetesService** pour le déploiement automatique de leurres (honeypots) en réponse à des menaces détectées.

5. Gestion de Projet & Qualité

5.1. Sécurité & Authentification

L'accès est strictement contrôlé pour protéger la progression des utilisateurs.

- **Inscription & Connexion :** Formulaires sécurisés avec gestion des erreurs (401/403).
- **Sécurisation des mots de passe :** Hachage robuste via Spring Security (BCrypt) avant stockage en base.
- **Protection des Routes (AuthGuard) :** Mécanisme Frontend empêchant l'accès direct aux pages sensibles (Dashboard, Arena) sans token de session valide.
- **Persistance :** Utilisation du `localStorage` pour une expérience utilisateur fluide.

5.2. Qualité du Code (CI/CD)

Nous avons industrialisé notre processus de développement pour la v1.0.

- **Pipeline Automatisé :** Chaque modification du code déclenche une compilation et des tests via GitHub Actions.
- **Tests Unitaires (JUnit) :** Couverture de la logique métier critique (Calcul de score, Génération de logs).
- **Tests E2E (Selenium) :** Validation automatique des parcours utilisateurs complets (Login → Arena → Fin).

- **Analyse Statique :** Zéro "Security Hotspot" critique sur SonarCloud.

6. Conclusion & Perspectives

La version v1.0.0 de CyberScale est une plateforme complète et innovante pour l'évaluation des compétences en cybersécurité. En combinant théorie, pratique, et une architecture technique avancée, nous offrons une expérience utilisateur unique.